VGGNet y GoogLeNet

Profundidad vs. eficiencia en la clasificación de imágenes a gran escala
Basado en los artículos originales (Simonyan & Zisserman, 2015; Szegedy et al., 2015)

Universidad Nacional de Ingeniería — FIIS, Unidad de Posgrado
Redes Neuronales y Aprendizaje Profundo (Clases 4 y 5)

Sobre este documento

Estas notas resumen, en las propias palabras del curso, el contenido de dos artículos que compitieron —y definieron— el ILSVRC-2014: *Very Deep Convolutional Networks for Large-Scale Image Recognition* (K. Simonyan y A. Zisserman, ICLR 2015; arXiv:1409.1556) y *Going Deeper with Convolutions* (C. Szegedy et al., CVPR 2015; arXiv:1409.4842). Todas las tablas, cifras y citas numéricas provienen de esos artículos; no se reproduce texto textual de ellos. Se recomienda leer los originales para el detalle completo de los experimentos.

Índice

1. Contexto: ILSVRC-2014 **2**

2. VGGNet: Very Deep Convolutional Networks **2**

2.1. Motivación . . . 2
2.2. Arquitectura genérica . . . 2
2.3. Las configuraciones A–E . . . 2
2.4. ¿Por qué apilar filtros 3×3? . . . 3
2.5. Entrenamiento . . . 4
2.6. Evaluación: red totalmente convolucional . . . 4
2.7. Resultados principales . . . 4
2.8. Conclusión del artículo . . . 5

3. GoogLeNet: Going Deeper with Convolutions **5**

3.1. Motivación: la vía "ancha y profunda pero eficiente" . . . 5
3.2. El módulo Inception . . . 5
3.3. La arquitectura GoogLeNet . . . 6
3.4. Clasificadores auxiliares . . . 6
3.5. Entrenamiento . . . 7
3.6. Resultados . . . 7

4. VGGNet vs. GoogLeNet **8**

5. Resumen y preguntas de repaso **8**

1. Contexto: ILSVRC-2014

El ImageNet Large-Scale Visual Recognition Challenge de 2014 (ILSVRC-2014) fue el escenario donde compitieron ambas arquitecturas. VGG (equipo de Oxford) obtuvo el segundo lugar en clasificación con 7.32 % de error top-5, y GoogLeNet (equipo de Google) obtuvo el primer lugar con 6.67 %. Curiosamente, ambos equipos llegaron a conclusiones distintas sobre *cómo* lograr una red mejor:

	VGGNet	**GoogLeNet**
Estrategia	Profundidad uniforme, filtros 3×3	Módulos "Inception" multi-escala
Profundidad	16–19 capas con pesos	22 capas con pesos
Parámetros	138 M (config. D)	5 M (12× menos que AlexNet)
Filosofía	"Simple y profundo"	"Ancho y disperso, aproximado por bloques densos"

Esta clase presenta ambos artículos en sus propios términos y cierra comparándolos.

2. VGGNet: Very Deep Convolutional Networks

2.1. Motivación

El artículo de Simonyan y Zisserman fija todos los demás parámetros de diseño de la red y estudia sistemáticamente **un solo factor**: la profundidad, aumentada agregando más capas convolucionales con filtros muy pequeños, 3×3 en todas las capas. Es el tamaño más pequeño que aún captura la noción de izquierda/derecha, arriba/abajo, centro.

2.2. Arquitectura genérica

- Entrada: imagen RGB de tamaño fijo 224×224; el único preprocesamiento es restar la media RGB del conjunto de entrenamiento.
- Todas las convoluciones usan *stride* 1 y *padding* 1 (para 3×3), de modo que preservan la resolución espacial.
- Cinco capas de *max-pooling* (2×2, stride 2) reducen la resolución progresivamente; no todas las capas convolucionales van seguidas de pooling.
- Al final, tres capas totalmente conectadas: 4096, 4096 y 1000 (una por clase de ILSVRC), seguidas de softmax.
- Todas las capas ocultas usan ReLU. Ninguna red (salvo una, de control) usa Normalización de Respuesta Local (LRN): los autores muestran que no mejora el desempeño y sólo incrementa memoria y tiempo de cómputo.

2.3. Las configuraciones A–E

El ancho (número de canales) empieza en 64 en la primera capa y se **duplica** después de cada max-pooling hasta llegar a 512. La Tabla 1 reproduce la estructura del artículo (notación "conv⟨tamaño⟩-⟨canales⟩").

Cuadro 1: Configuraciones de VGGNet (A a E), de 11 a 19 capas con pesos.

A (11 capas)	A-LRN (11)	B (13)	C (16)	D (16)	E (19)
entrada (224×224 RGB)					
conv3-64	conv3-64+LRN	conv3-64 ×2		conv3-64 ×2	
maxpool					
conv3-128	conv3-128	conv3-128 ×2		conv3-128 ×2	
maxpool					
conv3-256 ×2	conv3-256 ×2	conv3-256 ×2	conv3-256×2 + conv1-256	conv3-256×3	conv3-256×4
maxpool					
conv3-512×2	conv3-512×2	conv3-512×2	conv3-512×2+conv1-512	conv3-512×3	conv3-512×4
maxpool					
conv3-512×2	conv3-512×2	conv3-512×2	conv3-512×2+conv1-512	conv3-512×3	conv3-512×4
maxpool					
FC-4096 — FC-4096 — FC-1000 — softmax					

VGG-16 es la configuración D (16 capas con pesos: 13 conv. + 3 FC); **VGG-19** es la E (16 conv. + 3 FC = 19). La configuración C introduce tres capas conv1 (convoluciones 1×1) mientras que D las reemplaza todas por conv3.

Cuadro 2: Número de parámetros por configuración (millones). Fuente: Tabla 2 del artículo.

Red	A / A-LRN	B	C	D	E
Parámetros (M)	133	133	134	138	144

2.4. ¿Por qué apilar filtros 3×3?

Este es el argumento central del artículo. Dos capas conv. 3×3 sin pooling entre ellas tienen un campo receptivo efectivo de 5×5; tres capas 3×3 equivalen a un campo receptivo de 7×7. Frente a usar directamente un filtro 7×7, apilar tres 3×3 ofrece dos ventajas:

1. **Más no linealidad.** Se incorporan tres funciones de rectificación (ReLU) en vez de una sola, haciendo la función de decisión más discriminativa.
2. **Menos parámetros.** Si la entrada y salida de la pila de tres capas tienen C canales, la pila de 3×3 usa
$$3\,(3^2C^2) = 27\,C^2 \tag{1}$$
parámetros, mientras que una sola capa 7×7 usaría $7^2C^2 = 49\,C^2$, es decir, un **81 % más** de parámetros. Esto equivale a imponer una regularización sobre el filtro 7×7, forzándolo a factorizarse a través de filtros 3×3 con no linealidad intercalada.

Las convoluciones 1×1 (configuración C) son una forma de aumentar la no linealidad de la función de decisión sin afectar el campo receptivo: son, en esencia, una proyección lineal seguida de ReLU (idea tomada de *Network in Network*, Lin et al., 2014).

Dato del artículo

Los autores compararon la red B con una variante que reemplaza cada par de capas 3×3 por una sola capa 5×5 (mismo campo receptivo). La red superficial con filtros 5×5 tuvo un error top-1 **7 % mayor** que B, confirmando que una red profunda con filtros pequeños supera a una red superficial con filtros grandes.

2.5. Entrenamiento

- Descenso de gradiente por mini-lotes con momento, tamaño de lote 256, momento 0.9.
- Weight decay (L_2) con multiplicador 5×10^{-4}; dropout (ratio 0.5) en las dos primeras capas FC.
- Tasa de aprendizaje inicial 10^{-2}, dividida por 10 cuando la exactitud de validación deja de mejorar (3 veces en total); entrenamiento detenido tras 370 K iteraciones (≈74 épocas).
- Inicialización: se entrena primero la red A (poco profunda) con inicialización aleatoria; las redes más profundas inicializan sus primeras 4 capas conv. y las 3 FC finales con los pesos de A (las capas intermedias se inicializan aleatoriamente, $\mathcal{N}(0, 10^{-2})$).
- **Aumento de escala de entrenamiento (*scale jittering*):** sea S el lado menor de la imagen reescalada de la que se recorta la entrada 224×224. Se probaron S fijo (256 o 384) y S muestreado aleatoriamente en $[256, 512]$ para cada imagen. El *jittering* multi-escala en entrenamiento mejora significativamente el resultado, incluso evaluando a una sola escala en test.

2.6. Evaluación: red totalmente convolucional

En el momento de evaluar, las capas FC se convierten en capas convolucionales (la primera FC en conv7×7, las dos últimas en conv1×1), de modo que la red se vuelve totalmente convolucional y puede aplicarse de forma *densa* sobre la imagen completa a cualquier resolución, produciendo un mapa de puntajes por clase que luego se promedia espacialmente (*sum-pooling*). Esto evita tener que recortar múltiples ventanas de la imagen en test, aunque los autores también evalúan con 150 recortes por imagen para comparar.

2.7. Resultados principales

Config.	Escala entren. S	Error top-1 (%)	Error top-5 (%)
A	256	29.6	10.4
B	256	28.7	9.9
D (VGG-16)	$[256, 512]$	25.6	8.1
E (VGG-19)	$[256, 512]$	25.5	8.0

El error decrece monótonamente con la profundidad (A→E) y se *satura* en 19 capas. La combinación (*ensemble*) de las redes D y E, con evaluación densa y multi-recorte, alcanza 6.8 % de error top-5 en validación —competitivo con GoogLeNet (6.7 %)— y el mejor resultado de una *sola* red (7.0 %), superando a una sola GoogLeNet por 0.9 puntos porcentuales. El equipo VGG ganó además la tarea de localización de ILSVRC-2014 con 25.3 % de error.

2.8. Conclusión del artículo

Sin apartarse de la arquitectura clásica de ConvNet (LeCun et al., 1989; Krizhevsky et al., 2012), los autores muestran que **aumentar sustancialmente la profundidad**, usando filtros pequeños en todas las capas, basta para alcanzar el estado del arte. Ese es su único grado de libertad de diseño.

3. GoogLeNet: Going Deeper with Convolutions

3.1. Motivación: la vía "ancha y profunda pero eficiente"

Szegedy et al. atacan el mismo problema —redes más grandes y precisas— pero identifican dos problemas de simplemente agrandar una red uniformemente (más capas, más filtros por capa):

1. **Sobreajuste:** más parámetros son más propensos a sobreajustar, especialmente con datos etiquetados limitados.
2. **Costo computacional:** si se encadenan dos capas convolucionales, cualquier incremento uniforme de sus filtros produce un incremento *cuadrático* del cómputo; si la capacidad añadida se usa de forma ineficiente, se desperdicia cómputo.

La solución de fondo, inspirada en el trabajo teórico de Arora et al. (2013) y en el principio Hebbiano ("neuronas que se activan juntas, se conectan juntas"), sería pasar de arquitecturas totalmente conectadas a **arquitecturas dispersas**, incluso dentro de las convoluciones: si la distribución de probabilidad de los datos es representable por una red dispersa muy grande, la topología óptima puede construirse capa por capa analizando las correlaciones de las activaciones y agrupando unidades altamente correlacionadas.

El problema práctico: el hardware actual es muy ineficiente para cálculo numérico sobre estructuras dispersas no uniformes (el costo de las búsquedas y fallos de caché domina, incluso reduciendo $100\times$ las operaciones aritméticas). La pregunta que motiva Inception es: *¿existe un paso intermedio que aproveche la dispersión sugerida por la teoría, a nivel de filtro, pero que se ejecute con matrices densas eficientes en el hardware actual?*

3.2. El módulo Inception

La idea: encontrar la construcción local óptima y repetirla espacialmente. Se asume que las unidades correlacionadas de una capa se concentran en regiones locales (cubribles con conv. 1×1) y que existe un número decreciente de clusters más dispersos espacialmente, cubribles con parches más grandes. Para evitar problemas de alineación de parches, el módulo se restringe a tamaños 1×1, 3×3 y 5×5, más una rama de *pooling* paralela (dado que el pooling ha sido esencial en el estado del arte). Los filtros de salida de todas las ramas se **concatenan**.

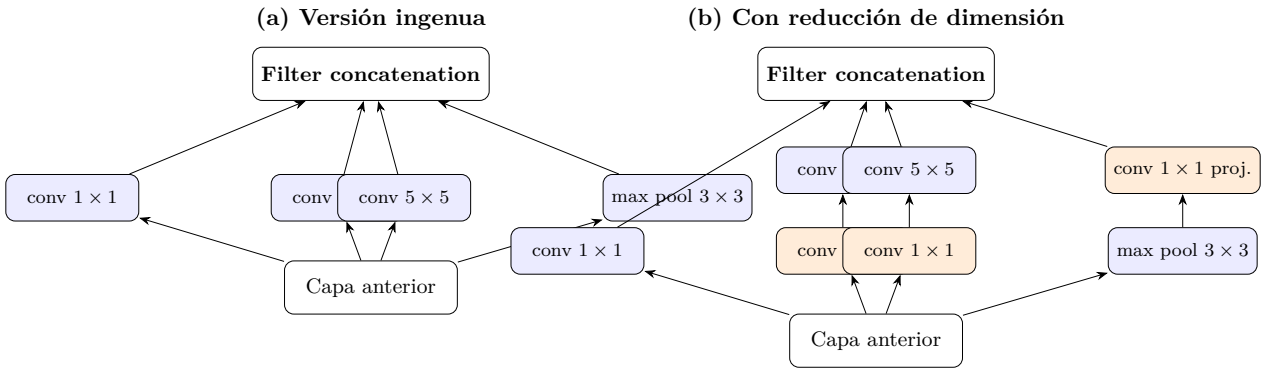


Figura 1. Módulo Inception: (a) versión ingenua; (b) con convoluciones 1×1 de reducción de dimensión antes de las convoluciones $3 \times 3 / 5 \times 5$ y después del pooling. Recreación esquemática de la Figura 2 de Szegedy et al. (2015).

El problema de la versión ingenua: incluso un número modesto de convoluciones 5×5 sobre una capa con muchos filtros es extremadamente costoso, y la salida del pooling conserva el mismo número de canales de la etapa anterior, de modo que el número de salidas crece de etapa en etapa —una explosión computacional en pocas etapas.

La solución: aplicar reducciones de dimensión mediante convoluciones 1×1 *antes* de las convoluciones 3×3 y 5×5 costosas, y una proyección 1×1 *después* del max-pooling. Estas convoluciones 1×1 cumplen doble función (idea de *Network in Network*, Lin et al., 2014): reducen dimensionalidad **y** incluyen una no linealidad ReLU. Esto permite incrementar el ancho y la profundidad de la red sin explosión de cómputo, y mantiene la representación dispersa en la mayoría de los lugares, comprimiendo la señal sólo cuando debe agregarse masivamente.

3.3. La arquitectura GoogLeNet

GoogLeNet es la instancia particular de la arquitectura Inception usada en la competencia (nombre en homenaje a LeNet-5 de LeCun). Tiene **22 capas con parámetros** (27 contando pooling), y del orden de 100 bloques independientes en total. Todas las convoluciones —incluidas las internas de los módulos Inception— usan ReLU. Por razones de eficiencia de memoria durante el entrenamiento, las primeras capas mantienen la forma convolucional tradicional y los módulos Inception se usan sólo en las capas superiores. Se usa *average pooling* antes del clasificador final (en vez de capas FC, como en *Network in Network*), lo que mejoró el top-1 en $\sim$0.6 puntos porcentuales; se agrega además una capa lineal para facilitar el ajuste fino a otros conjuntos de etiquetas.

"#3×3 red." y "#5×5 red." son el número de filtros 1×1 de la capa de reducción antes de las convoluciones 3×3 y 5×5; "pool proj" es el número de filtros 1×1 de proyección tras el max-pooling interno del módulo. **Resultado clave:** GoogLeNet usa **12 veces menos parámetros** que la red ganadora de 2012 (Krizhevsky et al.), siendo significativamente más precisa.

3.4. Clasificadores auxiliares

Dada la gran profundidad de la red, preocupaba propagar el gradiente de forma efectiva hasta las primeras capas. La intuición: si redes más superficiales ya funcionan razonablemente bien en esta tarea, las características de las capas intermedias deben ser bastante discriminativas. Por ello se añaden **dos clasificadores auxiliares**, conectados a la salida de los módulos Inception (4a) y (4d), cada uno compuesto por:

1. Average pooling 5×5, stride 3 ($\rightarrow 4 \times 4 \times 512$ en (4a), $4 \times 4 \times 528$ en (4d)).

Cuadro 3: GoogLeNet: la instancia concreta de Inception usada en ILSVRC-2014 (Tabla 1 del artículo).

Tipo	patch/stride	tamaño salida	#1×1	#3×3 red.	#3×3	#5×5 red.	#5×5	pool proj	params	ops
convolución	7×7/2	112×112×64							2.7K	34M
max pool	3×3/2	56×56×64								
convolución	3×3/1	56×56×192		64	192				112K	360M
max pool	3×3/2	28×28×192								
inception (3a)		28×28×256	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480								
inception (4a)		14×14×512	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832								
inception (5a)		7×7×832	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024								
dropout (40 %)		1×1×1024								
lineal		1×1×1000							1000K	1M
softmax		1×1×1000								

2. Convolución 1×1 con 128 filtros (reducción de dimensión) y ReLU.
3. Capa FC de 1024 unidades con ReLU.
4. Dropout con 70 % de tasa.
5. Capa lineal + softmax (mismas 1000 clases).

Durante el entrenamiento, la pérdida de cada clasificador auxiliar se suma a la pérdida total con un **peso de descuento de 0.3**. En inferencia, estas ramas auxiliares se descartan por completo. Su función: alentar la discriminación en las etapas bajas, aumentar la señal de gradiente que se propaga hacia atrás, y aportar regularización adicional.

3.5. Entrenamiento

Entrenada con DistBelief (sistema distribuido de Google, basado en CPU) usando SGD asíncrono con momento 0.9 y una programación de tasa de aprendizaje fija (decrece 4 % cada 8 épocas); se usó promediado de Polyak para el modelo final. En evaluación, se usó un esquema agresivo de recortes múltiples: 4 escalas × 3 posiciones × 6 recortes (4 esquinas + centro + imagen completa) × 2 (con espejo) = **144 recortes por imagen**; las probabilidades softmax se promedian sobre recortes y sobre los clasificadores del *ensemble*.

3.6. Resultados

Equipo	Año	Error top-5	Datos externos
SuperVision (Krizhevsky et al.)	2012	16.4 %	no
Clarifai	2013	11.7 %	no
MSRA	2014	7.35 %	no
VGG	2014	7.32 %	no
GoogLeNet	2014	**6.67 %**	no

Efecto del número de modelos y de recortes (sobre validación): 1 modelo/1 recorte da 10.07 % de error top-5; agregar recortes (hasta 144) lo reduce a 7.89 %; usar un *ensemble* de 7 modelos con

144 recortes cada uno alcanza el resultado final de **6.67 %**. GoogLeNet también ganó la tarea de detección de ILSVRC-2014, usando el módulo Inception como clasificador de regiones dentro de un esquema tipo R-CNN.

4. VGGNet vs. GoogLeNet

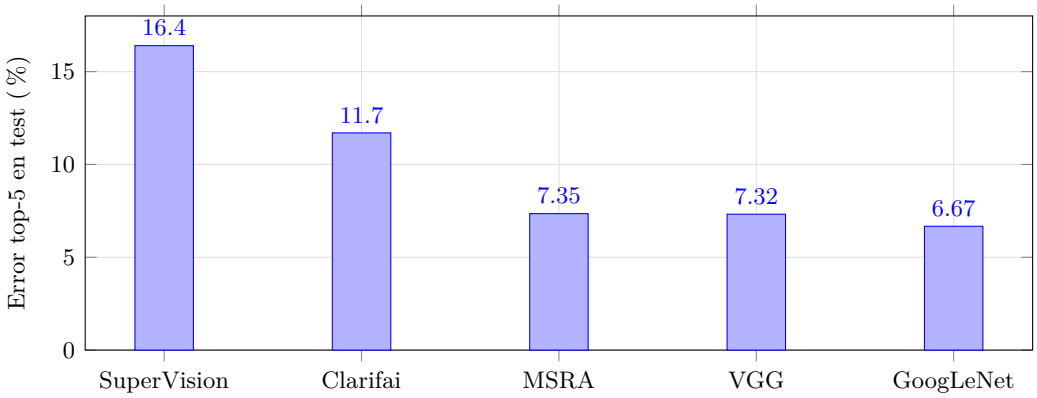


Figura 2. Error top-5 (test, sin datos externos) de los principales equipos de ILSVRC 2012–2014. Datos: Tabla 2 de Szegedy et al. (2015) y Tabla 7 de Simonyan & Zisserman (2015).

	VGGNet (config. D/E)	**GoogLeNet**
Profundidad	16–19 capas con pesos	22 capas con pesos
Parámetros	138–144 M	∼5–6 M
Bloque básico	pila uniforme de conv. 3×3	módulo Inception multi-escala
Reducción de dimensión	sólo implícita (conv1 en config. C)	explícita y sistemática (conv1 antes de 3×3/5×5)
Clasificador final	3 capas FC (4096-4096-1000)	average pooling global + 1 capa lineal
Regularización extra	dropout en FC	dropout + 2 clasificadores auxiliares
Filosofía de diseño	profundidad uniforme, simplicidad	aproximar una estructura dispersa óptima con bloques densos
Error top-5 (test, ILSVRC-14)	7.32 % (7 modelos) / 7.0 % (1 modelo)	6.67 % (7 modelos)

Lo que dice cada artículo del otro

Simonyan y Zisserman señalan que GoogLeNet (desarrollado independientemente) también se basa en ConvNets muy profundas (22 capas) y filtros pequeños, pero con una topología más compleja y una reducción de resolución más agresiva en las primeras capas; su propio modelo, sin embargo, supera a GoogLeNet en exactitud de una sola red. Ambos artículos coinciden en el mensaje central: **la profundidad importa**, aunque difieren radicalmente en cómo pagar su costo computacional — VGG lo acepta (138 M de parámetros), GoogLeNet lo evita por diseño (5 M de parámetros).

5. Resumen y preguntas de repaso

Ideas clave.

1. VGGNet demuestra que apilar convoluciones 3×3 (sin cambiar nada más) y aumentar la

profundidad hasta 16–19 capas basta para mejorar sustancialmente sobre el estado del arte previo.

2. Apilar n capas 3×3 reproduce el campo receptivo de una capa $(2n{+}1) \times (2n{+}1)$ con más no linealidad y menos parámetros.
3. GoogLeNet ataca el problema de escalar redes mediante módulos Inception: múltiples tamaños de filtro en paralelo, concatenados, con convoluciones 1×1 que reducen dimensión *antes* de las operaciones costosas.
4. Los clasificadores auxiliares combaten el desvanecimiento del gradiente en redes muy profundas y añaden regularización; se descartan en inferencia.
5. GoogLeNet logra mejor exactitud que VGG con $\sim$25 veces menos parámetros, mostrando que el diseño del bloque importa tanto como la profundidad.

Preguntas de repaso.

1. Calcule cuántos parámetros ahorra una pila de tres convoluciones 3×3 frente a una única convolución 7×7, para $C = 256$ canales de entrada y salida.
2. ¿Por qué la configuración C (con conv1×1) es peor que la D (con conv3×3) a pesar de tener la misma profundidad? Relacione su respuesta con el concepto de campo receptivo.
3. Explique el problema de la "explosión computacional" del módulo Inception ingenuo y cómo lo resuelven las convoluciones 1×1 de reducción.
4. ¿Por qué los clasificadores auxiliares se ponderan por 0.3 durante el entrenamiento y se descartan en inferencia?
5. Compare el número de parámetros de VGG-16 (138 M) y GoogLeNet ($\sim$5 M). ¿Qué decisiones de diseño explican una diferencia de casi 30×?
6. ¿Qué rol cumple el *average pooling* global en GoogLeNet en comparación con las capas totalmente conectadas de VGG?

Referencias

Simonyan, K., & Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. *ICLR*. arXiv:1409.1556.

Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2015). Going Deeper with Convolutions. *CVPR*. arXiv:1409.4842.

Lin, M., Chen, Q., & Yan, S. (2014). Network In Network. *ICLR*.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *NeurIPS*.

Arora, S., Bhaskara, A., Ge, R., & Ma, T. (2013). Provable Bounds for Learning Some Deep Representations. arXiv:1310.6343.

LeCun, Y., Boser, B., Denker, J. S., et al. (1989). Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation*.